BIG DATA MINI PROJECT | IMPLEMENTATION PLAN

Open-Source Pulse

Predicting trending GitHub repositories early with Spark, Parquet, GraphFrames and MLlib

Dataset: GH Archive (every public GitHub event, hourly)
Duration: 6 weeks | Prepared: October 2026

At a glance	
Question	Which new GitHub repositories will gain a large number of stars in the next 30 days, and can we flag them weeks before they trend?
Data	GH Archive hourly event files (stars, forks, pull requests, issues, pushes)
Core stack	PySpark, Parquet (zstd), GraphFrames, Spark MLlib, MLflow, FastAPI, DuckDB
Output	Weekly ranked list of predicted breakout repos, backtest results, efficiency benchmarks, live dashboard

Contents

1. Project overview 4

Research questions . . . 4

Why this is a strong big data project . . . 4

Scope . . . 4

2. Dataset: GH Archive 4

Event types used . . . 4

Choosing the data window . . . 5

Storage layout . . . 5

3. Architecture 6

4. Technology stack 6

Environment setup . . . 7

5. Key design decisions 7

6. Step-by-step implementation 7

Phase 1: Acquire the data . . . 7

Phase 2: Ingest to Bronze . . . 8

Phase 3: Clean to Silver . . . 8

Phase 4: Aggregate to Gold (repo-day, repo-week) . . . 9

Phase 5: Feature engineering . . . 9

Phase 6: Labels and time-based splits . . . 10

Phase 7: Graph analytics with GraphFrames . . . 10

Phase 8: Modelling with Spark MLlib . . . 11

Phase 9: Evaluation and backtest . . . 12

Phase 10: Serving and dashboard . . . 12

7. Efficiency playbook 13

Starting Spark configuration (local 16 GB machine) . . . 13

Common pitfalls . . . 14

8. Benchmark study 14

9. Repository structure 15

10. Six-week timeline 15

11. Risks and mitigations 15

12. Presentation plan 16

Suggested slide flow (12 slides) . . . 16

Questions to rehearse . . . 16

13. Deliverables and stretch goals 17

Final deliverables checklist . . . 17

Stretch goals . . . 17

Ethics and compliance . . . 17

Appendix: quick-start order . . . 17

1. Project overview

Stars are the most visible signal of open-source momentum, but by the time a repository appears on a trending page the early advantage is gone. This project builds a pipeline that processes GitHub's public event history at scale, learns what the first weeks of a breakout look like, and ranks new repositories by their probability of a large star gain over the next 30 days.

Research questions

- **RQ1.** Which early signals (star velocity, fork ratio, pull-request activity, audience composition) best predict a breakout?
- **RQ2.** How much lead time does the model give compared with simple threshold rules?
- **RQ3.** Does graph context (momentum of repos that share stargazers) add predictive power beyond per-repo features?
- **RQ4.** How do engineering choices (file format, partitioning, aggregation grain, caching) change runtime as data grows?

Why this is a strong big data project

- **Real scale.** The raw archive is billions of events; a single hour of data is already a non-trivial file, and a multi-month window cannot be handled with pandas.
- **Several big data concepts in one pipeline.** Distributed ingestion, columnar storage and partitioning, shuffles and skew, window functions, graph analytics, distributed ML.
- **Measurable efficiency.** Every optimisation in Section 7 can be benchmarked (Section 8), which gives you evidence rather than claims.
- **Demo-friendly.** A ranked list of predicted breakouts with a backtest is easy to explain and easy to question live.

Scope

In scope	Out of scope (listed as stretch goals)
Batch pipeline over a chosen time window; repo-level prediction; co-star graph features; backtest on past breakouts; benchmark study; REST API and dashboard	True real-time streaming; README or code text analysis; GitHub REST metadata enrichment (language, topics); deep learning models

> **Honest framing for your presentation.** This predicts *star growth*, which is a popularity signal, not code quality. Say so up front; it makes the project more credible, not less.

2. Dataset: GH Archive

GH Archive records the public GitHub timeline and publishes it as one gzipped JSON file per hour, one event per line. File names follow the pattern `https://data.gharchive.org/YYYY-MM-DD-H.json.gz` where the hour H runs from 0 to 23 without zero padding. The same data is also available as public tables in BigQuery.

Event types used

Event type	What it means	Used for
`WatchEvent`	A user starred a repository (payload action is 'started')	Target label and velocity features
`ForkEvent`	A repository was forked (the event's repo is the original)	Adoption signal, fork-to-star ratio
`CreateEvent`	Branch, tag or repository created; ref_type = 'repository' marks a new repo	Repo age and exact cumulative counts
`PullRequestEvent`	Pull request opened, closed or merged	Collaboration and maintainer responsiveness

Event type	What it means	Used for
`IssuesEvent`	Issue opened or closed	User engagement
`PushEvent`	Commits pushed to a repo	Maintainer activity (largest event type, see note)

PushEvent is typically the biggest share of all events. Keep it in the first run because commit activity is a useful signal, but measure its cost in the benchmark study; dropping it is the single biggest cheap speed-up if you need one.

Choosing the data window

Do not guess the size. Before committing to a window, download one representative day (24 files), note its compressed size, and multiply by the number of days. Plan in tiers so every stage can be developed on small data and then scaled:

Tier	Window	Purpose	Where to run
T0	1 week	Schema profiling, unit tests, fast iteration	Laptop
T1	3 months	End-to-end pipeline, first model	Laptop (16 GB RAM, 8 cores, SSD)
T2	12 to 18 months (main result)	Final training, backtest, ablations	Large single VM or small cluster
T3	Full multi-year window (stretch)	Scaling curve, headline numbers	Cluster (3 to 5 workers for a few hours)

Pick the T2 window so it contains several well-known breakout repositories (check star-history charts for repos created inside your window). You will use them as the backtest showpiece in Section 6, Phase 9.

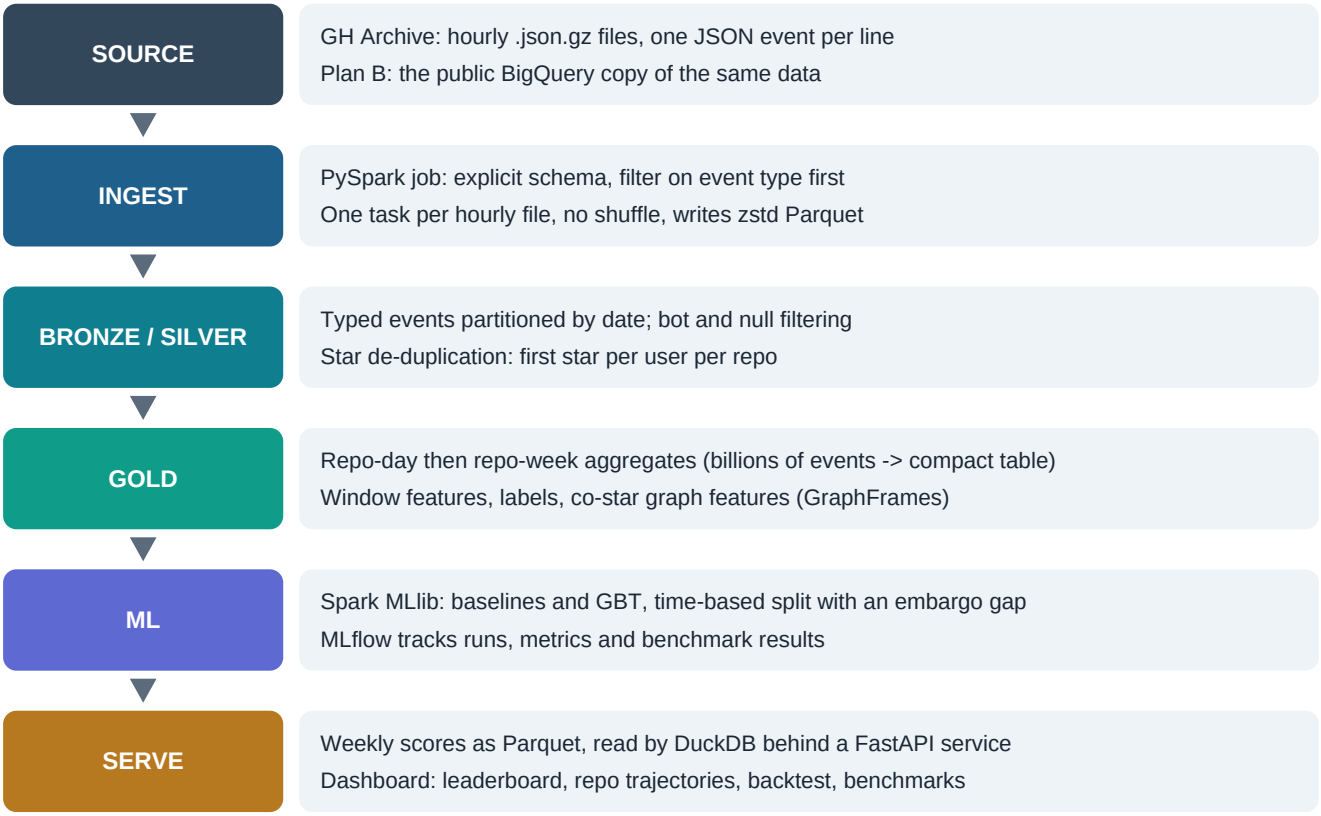
Storage layout

```
data/
  raw/                              # hourly .json.gz (deleted after conversion, keep a manifest)
  bronze/events/dt=YYYY-MM-DD/      # filtered, typed events (Parquet, zstd)
  silver/events/                    # bot-filtered events, compacted by month, sorted by repo_id
  gold/repo_day/                    # one row per repo per day
  gold/repo_week/                   # one row per repo per week (feature and label source)
  gold/features/                    # training table with labels
  gold/graph/                       # monthly co-star graph features
  serving/scores/                   # weekly model scores for the API
```

Bandwidth and disk are real constraints. Process month by month: download, convert to Parquet, delete the raw files. Bronze is a small fraction of raw size, so you never need the whole archive on disk at once. If your connection is the bottleneck, Plan B is to extract only the columns you need from the BigQuery copy and continue in Spark from there; mention the trade-off in your report.

3. Architecture

The pipeline follows a medallion layout. Each layer shrinks the data, so the expensive operations (windows, joins, graph work, model training) always run on the smallest table that can answer the question.



Layer	Grain	Relative size	Format
Raw	One event per line	Largest	JSON, gzip
Bronze	One event (selected fields only)	Much smaller	Parquet, partitioned by date
Silver	One event, cleaned	Slightly smaller	Parquet, monthly, sorted by repo_id
Gold: repo-day	Repo x day	Orders of magnitude smaller	Parquet
Gold: repo-week	Repo x week (candidates only)	Small enough for one machine to reason about	Parquet

Record the actual row counts at each layer in your report (a 'funnel' table). It is the clearest single picture of why the design is efficient.

4. Technology stack

Concern	Choice	Why
Compute	PySpark 3.5.x on Java 17	Stable, and the version most third-party packages (GraphFrames, XGBoost-Spark) target. Verify current compatibility before you start.
Storage	Parquet with zstd	Columnar, splittable, strong compression, supports column pruning and predicate pushdown
Graph	GraphFrames	DataFrame-based graph API on Spark: label propagation, connected components, PageRank. Match the package version to your Spark and Scala build.
ML	Spark MLlib (GBTClassifier, LogisticRegression)	Distributed training inside the same engine; optional XGBoost on Spark as a stretch
Experiment tracking	MLflow	Parameters, metrics, artifacts and benchmark runs in one place

Concern	Choice	Why
Baseline engines	DuckDB and pandas	Honest comparison on one month of data: shows when Spark helps and when it does not
Serving	FastAPI + DuckDB, React or Streamlit dashboard	Serve precomputed scores in milliseconds; Spark stays offline
Packaging	Docker, Makefile	One-command reproducibility for the API, dashboard and pipeline steps

Environment setup

```
sudo apt install -y openjdk-17-jdk aria2          # Java for Spark, parallel downloader
python3 -m venv .venv && source .venv/bin/activate
pip install "pyspark>=3.5,<3.6" mlflow fastapi uvicorn duckdb pandas pyarrow plotly
# GraphFrames is added at submit time, for example:
#   --packages graphframes:graphframes:0.8.3-spark3.5-s_2.12   (check the current release)
```

5. Key design decisions

Settle these before writing code; they determine correctness and cost.

Decision	Recommended starting point	Reasoning
Prediction unit	One row per (repo, week)	Weekly snapshots keep the table small and match how people follow trends
Candidate pool	Repos created inside the window, with 20 to 300 stars at the snapshot and age up to 26 weeks	'Early' detection. Restricting to repos created in the window makes cumulative counts exact without needing history before the window
Label	1 if the repo gains at least 500 stars in the next 4 weeks (about 30 days)	Clear, countable, tunable. Check the positive rate and adjust the 500 threshold after profiling
Split	Strictly by time: train, 4-week gap, validation, 4-week gap, test	The gap stops label windows overlapping across splits (leakage)
Metric	PR-AUC and precision at K per week	The positive class is rare; ROC-AUC hides that
Bots and spam	Filter bot accounts, add burstiness and new-account share as features	Star campaigns exist; do not let them define 'trending'

All thresholds here are starting values. Plot the distribution of next-4-week star gains first, then choose the cut-off so the positive class is rare but large enough to learn from.

6. Step-by-step implementation

Phase 1: Acquire the data

Generate the URL list for one month, download in parallel with resume support, then verify that every day has 24 files. GH Archive occasionally has gaps; log missing hours in a manifest instead of failing.

```
# scripts/make_urls.sh  START END  (GNU date; on Windows use WSL)
d="$1"; end=$(date -I -d "$2 + 1 day")
while [ "$d" != "$end" ]; do
  for h in $(seq 0 23); do echo "https://data.gharchive.org/${d}-${h}.json.gz"; done
  d=$(date -I -d "$d + 1 day")
done > urls.txt

aria2c -i urls.txt -d data/raw -j 8 -x 2 -c      # 8 files at once, resumable
ls data/raw | cut -c1-10 | sort | uniq -c | awk '$1 != 24'   # days with missing hours
```

Phase 2: Ingest to Bronze

Read the hourly files with an **explicit schema** that lists only the fields you need. Spark's JSON reader then skips schema inference and does not materialise the rest of the payload. Because gzip files are not splittable, each hourly file becomes one task, which gives natural parallelism with no shuffle.

```
# jobs/01_ingest.py
import sys
from pyspark.sql import SparkSession, functions as F, types as T

KEEP = ["WatchEvent", "ForkEvent", "CreateEvent", "PullRequestEvent", "IssuesEvent", "PushEvent"]
SCHEMA = T.StructType([
    T.StructField("id", T.StringType()),
    T.StructField("type", T.StringType()),
    T.StructField("created_at", T.StringType()),
    T.StructField("actor", T.StructType([T.StructField("id", T.LongType()),
                                         T.StructField("login", T.StringType())])),
    T.StructField("repo", T.StructType([T.StructField("id", T.LongType()),
                                        T.StructField("name", T.StringType())])),
    T.StructField("org", T.StructType([T.StructField("id", T.LongType())])),
    T.StructField("payload", T.StructType([
        T.StructField("action", T.StringType()),
        T.StructField("ref_type", T.StringType()),
        T.StructField("size", T.LongType()),
        T.StructField("pull_request", T.StructType([T.StructField("merged", T.BooleanType())]))])),
])

def main(glob_in, out):
    spark = (SparkSession.builder.appName("pulse-ingest")
             .config("spark.sql.sources.partitionOverwriteMode", "dynamic").getOrCreate())
    raw = spark.read.schema(SCHEMA).json(glob_in)            # no inference pass
    ev = (raw.where(F.col("type").isin(KEEP))                # filter before anything else
          .select("id", "type", F.to_timestamp("created_at").alias("ts"),
                  F.col("repo.id").alias("repo_id"), F.col("repo.name").alias("repo_name"),
                  F.col("actor.id").alias("actor_id"), F.col("actor.login").alias("actor_login"),
                  F.col("org.id").alias("org_id"), F.col("payload.action").alias("action"),
                  F.col("payload.ref_type").alias("ref_type"),
                  F.col("payload.size").alias("push_size"),
                  F.col("payload.pull_request.merged").alias("pr_merged"))
          .withColumn("dt", F.to_date("ts")))
    (ev.write.mode("overwrite").partitionBy("dt")            # idempotent per-day re-runs
       .option("compression", "zstd").parquet(out))

if __name__ == "__main__":
    main(sys.argv[1], sys.argv[2])       # e.g. "data/raw/2024-01-*.json.gz" data/bronze/events
```

- **Schema drift.** An explicit schema returns nulls (not errors) if a field disappears in some year. Add a null-rate check per column per month and fail the job if it jumps.
- **Time zone.** Set `spark.sql.session.timeZone=UTC` so weekly boundaries are stable.
- **Idempotence.** Process one month per run; dynamic partition overwrite makes re-runs safe.

Phase 3: Clean to Silver

Keep this layer cheap. The aim is correctness on the few things that change the label, not a global clean-up pass.

Rule	Detail	Why
Drop null repo or actor ids	Filter rows where either is null	Cannot be attributed
Bot filter	Drop logins matching bot patterns (for example ending in [bot], dependabot, renovate, github-actions)	Automation inflates activity counts
Star de-duplication	Keep the first star per (repo_id, actor_id)	Unstar and re-star creates duplicate events and would inflate the label
No global dedupe on event id	Skip unless profiling shows real duplicates	A global distinct over billions of rows is a full shuffle for little gain
Compaction	Rewrite to monthly partitions sorted by repo_id	Fewer files, better row-group skipping

```
BOT = r"(\[bot\]$|-bot$|^dependabot|^renovate|^github-actions)"
ev = (spark.read.parquet("data/bronze/events")
```

```
       .where(F.col("repo_id").isNotNull() & F.col("actor_id").isNotNull())
       .where(~F.col("actor_login").rlike(BOT)))

stars = (ev.where(F.col("type") == "WatchEvent")
           .groupBy("repo_id", "actor_id").agg(F.min("ts").alias("ts"))   # first star only
           .withColumn("dt", F.to_date("ts")))
```

Phase 4: Aggregate to Gold (repo-day, repo-week)

This is the most important efficiency step: collapse events into per-repo daily counts, prune the long tail, then roll up to weeks. Window functions then run on a table that is orders of magnitude smaller.

```
# 1) lifetime stars per repo -> keep list of repos that can ever be candidates
star_tot = stars.groupBy("repo_id").agg(F.count("*").alias("n"))
keep = F.broadcast(star_tot.where("n >= 20").select("repo_id"))       # tiny vs. all repos

# 2) repos created inside the window (exact cumulative counts)
created = (ev.where((F.col("type") == "CreateEvent") & (F.col("ref_type") == "repository"))
             .groupBy("repo_id").agg(F.min("dt").alias("created_dt"),
                                     F.max("repo_name").alias("repo_name")))

# 3) repo-day aggregates, only for kept repos
star_day = stars.join(keep, "repo_id").groupBy("repo_id", "dt").agg(F.count("*").alias("stars"))
act_day = (ev.join(keep, "repo_id").where(F.col("type") != "WatchEvent")
   .groupBy("repo_id", "dt").agg(
       F.sum((F.col("type") == "ForkEvent").cast("int")).alias("forks"),
       F.sum(((F.col("type") == "PullRequestEvent") & (F.col("action") == "opened")).cast("int"))
        .alias("prs_opened"),
       F.sum(((F.col("type") == "PullRequestEvent") & F.col("pr_merged")).cast("int"))
        .alias("prs_merged"),
       F.sum((F.col("type") == "IssuesEvent").cast("int")).alias("issues"),
       F.sum((F.col("type") == "PushEvent").cast("int")).alias("pushes"),
       F.approx_count_distinct("actor_id").alias("actors")))
repo_day = (star_day.join(act_day, ["repo_id", "dt"], "full_outer").fillna(0)
            .join(created, "repo_id"))                          # candidates created in-window

# 4) repo-week with a numeric week index (Mondays; 2000-01-03 was a Monday)
repo_week = (repo_day
  .withColumn("week_start", F.to_date(F.date_trunc("week", "dt")))
  .withColumn("week_num", (F.datediff("week_start", F.lit("2000-01-03")) / 7).cast("long"))
  .groupBy("repo_id", "repo_name", "created_dt", "week_num", "week_start")
  .agg(F.sum("stars").alias("stars"), F.sum("forks").alias("forks"),
       F.sum("prs_opened").alias("prs_opened"), F.sum("prs_merged").alias("prs_merged"),
       F.sum("issues").alias("issues"), F.sum("pushes").alias("pushes"),
       F.max("stars").alias("max_day_stars"), F.countDistinct("dt").alias("active_days")))
repo_week.write.mode("overwrite").parquet("data/gold/repo_week")
```

Fill in this funnel table with your real counts; it becomes a slide.

Stage	Rows	Share of previous stage	Note
Raw events (all types)	____	-	Measured on your window
Bronze (selected types)	____	____	
Silver (clean)	____	____	
Repo-day (kept repos)	____	____	
Repo-week (candidates)	____	____	Feeds features and labels

Phase 5: Feature engineering

Window functions with `rangeBetween` over the numeric week index work on sparse repo-week rows, so you never have to materialise empty weeks. Every feature may only use weeks at or before the snapshot week.

Group	Features	Idea
Momentum	stars_w0 (this week), stars_w1 to stars_w3 (previous weeks), stars_4w, stars_cum, growth_ratio = w0 / (w1 + 1), acceleration, age_weeks	Speed and curvature of early growth

Group	Features	Idea
Adoption	forks_4w, fork_star_ratio	Forks suggest people are building on it, not just bookmarking
Collaboration	prs_opened_4w, prs_merged_4w, issues_4w, pushes_4w, active_days_4w	Healthy projects have maintainers responding
Burstiness	max_day_stars / stars_w0	One-day spikes (a front-page post, or a campaign) differ from steady growth
Audience	pct_stars_from_new_actors, pct_stars_from_heavy_stargazers	Computed on the pruned star table only; flags manufactured attention
Graph	costar_neighbour_growth, community_momentum, pagerank	Is the repo's neighbourhood heating up? (Phase 7)

```
from pyspark.sql import Window
w = Window.partitionBy("repo_id").orderBy("week_num")
def rs(col, lo, hi):                        # sum over a range of week offsets; null -> 0
    return F.coalesce(F.sum(col).over(w.rangeBetween(lo, hi)), F.lit(0))

feat = (spark.read.parquet("data/gold/repo_week")
  .withColumn("stars_cum",  rs("stars", Window.unboundedPreceding, 0))
  .withColumn("stars_w0",   F.col("stars"))
  .withColumn("stars_w1",   rs("stars", -1, -1))
  .withColumn("stars_w2",   rs("stars", -2, -2))
  .withColumn("stars_w3",   rs("stars", -3, -3))
  .withColumn("stars_4w",   rs("stars", -3, 0))
  .withColumn("forks_4w",   rs("forks", -3, 0))
  .withColumn("prs_opened_4w", rs("prs_opened", -3, 0))
  .withColumn("issues_4w",  rs("issues", -3, 0))
  .withColumn("pushes_4w",  rs("pushes", -3, 0))
  .withColumn("age_weeks",  F.datediff("week_start", "created_dt") / 7)
  .withColumn("growth_ratio", F.col("stars_w0") / (F.col("stars_w1") + 1))
  .withColumn("fork_star_ratio", F.col("forks_4w") / (F.col("stars_4w") + 1))
  .withColumn("burstiness", F.col("max_day_stars") / (F.col("stars_w0") + 1)))
```

Phase 6: Labels and time-based splits

```
H = 4                                                   # weeks ahead (about 30 days)
last_week = feat.agg(F.max("week_num")).first()[0]
data = (feat
  .withColumn("stars_next", rs("stars", 1, H))          # future window: label only, never a feature
  .where((F.col("stars_cum").between(20, 300)) & (F.col("age_weeks") <= 26))
  .where(F.col("week_num") <= last_week - H)            # drop weeks with incomplete futures
  .withColumn("label", (F.col("stars_next") >= 500).cast("int")))

# time split with an embargo gap equal to the label horizon
t1, t2 = <week_num cut 1>, <week_num cut 2>              # choose from the timeline
train = data.where(F.col("week_num") <= t1)
val   = data.where((F.col("week_num") > t1 + H) & (F.col("week_num") <= t2))
test  = data.where(F.col("week_num") > t2 + H)
```

Checks before modelling

- Print the positive rate per split. If it is far below 1 in 1,000, lower the star threshold or widen the candidate band.
- Confirm no feature column references `stars_next` or any week after the snapshot (write this as a unit test).
- Confirm no repo's label window overlaps a later split (the embargo gap above).

Phase 7: Graph analytics with GraphFrames

Build a **co-star graph**: two repos are linked when the same people starred both within a recent window. Dense neighbourhoods reveal ecosystems (for example, a cluster of related tools) and let you ask whether a repo's neighbours are heating up. A naive self-join is quadratic per user, so bound the work.

Guard	Effect
Drop heavy stargazers (for example more than 200 stars in the window)	Removes bots and star collectors, and the worst quadratic blow-ups

Guard	Effect
Restrict to kept repos and to the last 60 days	Shrinks the vertex set
Keep only edges with weight of at least 3 shared stargazers	Cuts noise and edge count
Recompute monthly, join to the most recent graph at or before each snapshot	Bounded cost and no leakage

```
from graphframes import GraphFrame
spark.sparkContext.setCheckpointDir("data/tmp/checkpoints")          # needed by components

recent = stars_recent.join(keep, "repo_id").join(heavy_actors, "actor_id", "left_anti")
pairs = (recent.alias("x").join(recent.alias("y"), "actor_id")
         .where(F.col("x.repo_id") < F.col("y.repo_id"))
         .groupBy(F.col("x.repo_id").alias("src"), F.col("y.repo_id").alias("dst"))
         .agg(F.count("*").alias("weight")).where("weight >= 3"))
rev = pairs.select(F.col("dst").alias("src"), F.col("src").alias("dst"), "weight")  # symmetric
edges = pairs.unionByName(rev)
verts = edges.select(F.col("src").alias("id")).distinct()
g = GraphFrame(verts, edges)

community = g.labelPropagation(maxIter=5)             # id, label
rank = g.pageRank(resetProbability=0.15, maxIter=10).vertices   # id, pagerank
# community_momentum = sum of stars_w0 of repos sharing a label at that snapshot
# costar_neighbour_growth = weighted mean growth_ratio over neighbours (join on edges)
```

Phase 8: Modelling with Spark MLlib

Start with the baselines, then the gradient-boosted trees. Handle the imbalance with a class weight; if training is slow, down-sample negatives and correct with weights. Cache the training DataFrame because MLlib iterates over it many times.

Model	Role
B0: rank by stars_w0	Naive baseline: 'whatever is getting stars now'
B1: rank by growth_ratio	Simple momentum rule
M1: logistic regression	Interpretable linear baseline
M2: GBTClassifier, per-repo features	Main model
M3: GBTClassifier, plus graph features	Tests RQ3

```
from pyspark.ml import Pipeline
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.classification import GBTClassifier
from pyspark.ml.evaluation import BinaryClassificationEvaluator
from pyspark.ml.functions import vector_to_array

FEATS = ["stars_cum", "stars_w0", "stars_w1", "stars_w2", "stars_w3", "growth_ratio",
         "forks_4w", "fork_star_ratio", "prs_opened_4w", "issues_4w", "pushes_4w",
         "burstiness", "age_weeks"]                    # + graph features for M3

train = train.cache()
pos = train.where("label = 1").count(); neg = train.count() - pos
train = train.withColumn("w", F.when(F.col("label") == 1, neg / pos).otherwise(1.0))

gbt = GBTClassifier(featuresCol="features", labelCol="label", weightCol="w",
                    maxDepth=6, maxIter=120, stepSize=0.1, subsamplingRate=0.8, seed=42)
pipe = Pipeline(stages=[VectorAssembler(inputCols=FEATS, outputCol="features",
                                        handleInvalid="keep"), gbt])
model = pipe.fit(train)
pred = model.transform(test).withColumn("score", vector_to_array("probability")[1])
pr_auc = BinaryClassificationEvaluator(labelCol="label", metricName="areaUnderPR").evaluate(pred)

# precision@K per week: how many of the weekly top-K were real breakouts?
wk = Window.partitionBy("week_num").orderBy(F.col("score").desc())
topk = pred.withColumn("rk", F.row_number().over(wk)).where(F.col("rk") <= 50)
p_at_50 = topk.groupBy("week_num").agg(F.avg("label").alias("precision_at_50"))
```

Tune on the validation split only (a small grid over `maxDepth`, `maxIter`, `stepSize`), log every run to MLflow, and touch the test split once at the end.

Phase 9: Evaluation and backtest

Evaluation	How
Ranking quality	PR-AUC (headline), precision@K and recall@K for K = 20, 50, 100 per week; ROC-AUC as a secondary number
Lead time	For each true breakout, weeks between the model's first top-K flag and the week it crossed a visible threshold (for example 1,000 stars). This is the number that sells the project.
Ablation	B0, B1, M1, M2, M3 side by side. Report what graph features and burstiness add.
Showpiece backtest	Pick 5 to 10 known breakouts inside your window. Plot their star curves with the first-flag week marked, and show the ranks they held.
Error analysis	False positives: one-day spikes and suspected star campaigns. False negatives: slow-burn projects that grew steadily rather than explosively.
Feature importance	Model importances from MLlib; SHAP on a sampled subset exported to pandas if time allows

Be upfront about drift. Open-source attention shifts with technology waves. Report performance by time period so a reader can see whether the model degrades on later months.

Phase 10: Serving and dashboard

Write weekly scores to a small Parquet table and let the API read it with DuckDB. Never call Spark from the API; the point of the offline pipeline is that serving is instant.

```
# api/main.py   (run: uvicorn api.main:app --reload)
import duckdb
from fastapi import FastAPI

app = FastAPI(title="Open-Source Pulse")
con = duckdb.connect()

@app.get("/top")
def top(week: str, k: int = 50):
    q = '''SELECT repo_name, score, stars_cum, stars_w0, growth_ratio
           FROM read_parquet('data/serving/scores/*.parquet')
           WHERE week_start = CAST(? AS DATE) ORDER BY score DESC LIMIT ?'''
    return con.execute(q, [week, k]).fetchdf().to_dict("records")

@app.get("/repo/{owner}/{name}")
def repo(owner: str, name: str):
    q = '''SELECT week_start, stars, stars_cum, score
           FROM read_parquet('data/serving/scores/*.parquet')
           WHERE repo_name = ? ORDER BY week_start'''
    return con.execute(q, [f"{owner}/{name}"]).fetchdf().to_dict("records")
```

Dashboard page	Content
Leaderboard	Top predicted repos for a selected week, score, current stars, momentum sparkline
Repo detail	Star curve, feature values, score history, nearest co-star neighbours
Backtest	Precision@K by week, lead-time distribution, showpiece breakouts
Pipeline	Funnel of row counts and the benchmark charts from Section 8

7. Efficiency playbook

Each technique below is paired with the evidence you should collect, so efficiency is something you demonstrate.

#	Technique	Where	What it buys	Evidence to show
1	Explicit schema, only needed fields	Ingest	No inference pass; the JSON reader skips unneeded payload	Ingest time with vs. without
2	Filter on event type first	Ingest	Drops most rows before any other work	Row counts before and after
3	Parquet with zstd	All layers	Column pruning, predicate pushdown, small files on disk	Size and scan time vs. JSON
4	One task per hourly gz file, no shuffle at ingest	Ingest	gzip is not splittable, so the file layout supplies parallelism	Throughput in GB per minute
5	Partition by date, sort by repo_id	Bronze, silver	Partition pruning on date ranges and row-group skipping	Query time with and without
6	Pre-aggregate before windowing	Gold	Windows run on repo-week rows, not events	Runtime and shuffle size
7	Prune long-tail repos, broadcast the keep list	Gold, graph	Removes most repos that never matter; avoids shuffling big tables	Row counts, plan showing BroadcastHashJoin
8	Dedupe only what changes the label	Silver	Avoids a global distinct over all events	Stage time saved
9	Adaptive query execution	All	Auto-coalesces shuffle partitions and splits skewed ones (viral repos)	Spark UI stage task-time spread
10	Built-in functions, no Python UDFs	All	Stay in the JVM engine; no serialisation to Python	Same job with a UDF, timed
11	Cache only reused DataFrames, then unpersist	ML	Avoids recomputing the training set each boosting round	Training time cached vs. not
12	Right-size partitions (about 128 to 256 MB; 2 to 4 tasks per core)	All	Avoids tiny-task overhead and huge-task spill	Shuffle partition sweep
13	Bound graph work: degree cap, edge-weight floor, monthly snapshots	Graph	Prevents quadratic pair explosion	Edge counts and runtime
14	Month-by-month batches with idempotent overwrite	Pipeline	Restartable, bounded disk use	Re-run after a forced failure
15	Serve precomputed scores from DuckDB	Serving	Millisecond API with no cluster	API latency measurement

Starting Spark configuration (local 16 GB machine)

```
spark-submit --master "local[8]" --driver-memory 12g \
  --conf spark.local.dir=/fast-ssd/spark-tmp \
  --conf spark.sql.session.timeZone=UTC \
  --conf spark.sql.shuffle.partitions=200 \
  --conf spark.sql.adaptive.enabled=true \
  --conf spark.sql.adaptive.coalescePartitions.enabled=true \
  --conf spark.sql.adaptive.skewJoin.enabled=true \
  --conf spark.sql.autoBroadcastJoinThreshold=64m \
  --conf spark.sql.files.maxPartitionBytes=134217728 \
  --conf spark.sql.parquet.compression.codec=zstd \
  --conf spark.sql.sources.partitionOverwriteMode=dynamic \
  --packages graphframes:graphframes:0.8.3-spark3.5-s_2.12 \
  jobs/04_graph.py
```

In local mode the driver and executors share one JVM, so `--driver-memory` is your total heap. On a cluster, set executor memory and cores instead, and keep executors to a moderate core count each. Treat these values as a starting point and tune them with the experiments in Section 8.

Common pitfalls

Pitfall	Symptom	Fix
Too many small files	Slow listing, thousands of tiny tasks	Compact to monthly partitions; coalesce before write
Calling collect() or toPandas() on large data	Driver out of memory	Aggregate first; only collect small results
count() after every step	Pipeline runs many times over	Count only when you log funnel numbers; cache if reused
Skew from viral repos	A few straggler tasks in a stage	Aggregate to repo-day before joins; rely on AQE; salt if needed
Join key explosion	Shuffle write in the hundreds of GB	Degree cap and edge-weight floor in the graph step
Label leakage	Test metrics that look too good	Time split, embargo gap, unit test on feature windows

8. Benchmark study

Run each experiment on the same machine, repeat three times, report the median, and record results in `benchmarks/results.csv`. Use the Spark UI (stages, shuffle read and write, spill) to explain the differences, not just to report them. Keep wall-clock timing around an action such as a write, since Spark evaluates lazily.

ID	Experiment	Compare	Metric
E1	Storage format	gz JSON vs. Parquet (snappy) vs. Parquet (zstd)	Size on disk, scan time
E2	Schema handling	Inferred schema vs. explicit partial schema	Ingest time
E3	Pruning	Select all columns vs. needed columns; date filter vs. none	Bytes read, query time
E4	Aggregation grain	Windows over events vs. over repo-week	Runtime, shuffle size
E5	Adaptive execution and partitions	AQE on and off; shuffle partitions 50, 200, 800	Runtime, straggler tasks
E6	Scaling with data	1 week, 1 month, 3 months, 12 to 18 months	Runtime vs. data size curve
E7	Scaling with cores	1, 2, 4, 8 cores on a fixed month	Speed-up curve
E8	Engine comparison	Spark vs. DuckDB vs. pandas on one month	Runtime, peak memory; where each stops working
E9	Graph guards	With and without degree cap and weight floor	Edge count, runtime

Results template

Experiment	Baseline (s)	Optimised (s)	Speed-up	Explanation from Spark UI
E1	____	____	____	
E4	____	____	____	
E6 (largest tier)	-	____	-	
E8 (Spark vs. DuckDB)	____	____	____	

Include E8 even if it is uncomfortable. On a single month, DuckDB may beat Spark. Showing where each tool wins, and that Spark's advantage appears as data or cluster size grows, is exactly the kind of understanding examiners look for.

9. Repository structure

```
open-source-pulse/
  README.md                 # how to run end to end
  Makefile                  # make download | ingest | silver | gold | features | train | serve
  config/pipeline.yaml      # window, thresholds, paths, Spark settings
  scripts/make_urls.sh
  jobs/
    01_ingest.py            # raw JSON -> bronze Parquet
    02_silver.py            # clean, star dedupe, compaction
    03_gold.py              # repo-day, repo-week
    04_graph.py             # co-star graph features (GraphFrames)
    05_features_labels.py   # windows, labels, splits
    06_train_eval.py        # baselines, GBT, MLflow logging
    07_score.py             # weekly scores for serving
  benchmarks/
    run_experiments.py      # E1..E9 harness
    results.csv
  api/main.py               # FastAPI + DuckDB
  dashboard/                # React or Streamlit app
  tests/                    # leakage test, schema test, small-fixture end-to-end test
  docker/                   # Dockerfiles for api and dashboard
  docs/                     # report, slides, architecture diagram
```

Create a tiny fixture dataset (a few hundred events with a hand-built 'breakout' repo) and run the full pipeline on it in the test suite. It catches window and leakage bugs in seconds instead of hours.

10. Six-week timeline

Week	Focus	Tasks	Exit criteria
1	Setup and data	Repo, environment, Makefile; profile one day; download and ingest T0 then T1; schema and null-rate checks	Bronze for 3 months; data profile notebook
2	Silver and gold	Bot filter, star dedupe, compaction; repo-day and repo-week; candidate pruning; funnel table	Gold tables; funnel numbers; first Spark UI screenshots
3	Features and labels	Window features; label and candidate definition; positive-rate study; time splits; leakage test	Feature table; leakage test passing
4	Modelling and scale-up	Baselines, MLflow, GBT and tuning; start T2 ingestion in the background	Model metrics on T1; T2 bronze complete
5	Graph, backtest, benchmarks	Co-star graph and features; ablation; showpiece backtest; run E1 to E9	Results tables and charts
6	Serving and presentation	FastAPI, dashboard, Docker; report; slides; demo rehearsal with a recorded backup	Working demo; final report and slides

If you run short on time: drop the graph features (Phase 7) and E9, and keep everything else. The project still answers RQ1, RQ2 and RQ4 with a complete pipeline and a strong benchmark study.

11. Risks and mitigations

Risk	Likelihood	Mitigation
Data volume exceeds disk or bandwidth	High	Tiered windows, month-by-month convert-and-delete, Plan B via BigQuery extraction
Positive class too rare to learn from	Medium	Profile gain distribution first; adjust star threshold and candidate band; class weights
Label leakage inflates results	Medium	Time split with embargo; automated leakage test; present validation and test separately
Fake or campaign stars distort the target	Medium	Bot filter, burstiness and new-account features; discuss as a limitation
GraphFrames version or memory problems	Medium	Pin versions early; test on T0; cut the graph step if blocked (see timeline note)

Risk	Likelihood	Mitigation
Model degrades over time (topic drift)	Medium	Report metrics by period; discuss retraining cadence
Live demo fails	Low to medium	Precomputed scores, recorded backup video, local Docker image

12. Presentation plan

Suggested slide flow (12 slides)

#	Slide	Key content
1	Problem	Why early detection of open-source momentum matters
2	Dataset	GH Archive: what it contains and how big your window is
3	Architecture	The layered diagram from Section 3
4	Data funnel	Row counts from raw to repo-week
5	Features and label	What 'breakout' means; candidate definition; leakage safeguards
6	Graph idea	Co-star graph picture and what communities looked like
7	Models	Baselines vs. GBT; why these
8	Results	PR-AUC and precision@K table; ablation
9	Backtest	Star curves with first-flag markers and lead time
10	Efficiency	Benchmark charts E1, E4, E6, E8
11	Live demo	Dashboard leaderboard and one repo drill-down
12	Limits and future work	Popularity is not quality; drift; streaming and richer metadata

Questions to rehearse

Likely question	Where your answer comes from
Why Spark and not pandas?	Experiment E8 and E6: where single-machine tools stop and how runtime scales
How do you prevent leakage?	Weekly snapshots, features only from past weeks, time split with embargo, the unit test
What about fake stars?	Bot filter, burstiness and new-account features; stated limitation
Why gradient-boosted trees?	Tabular, non-linear, handles mixed scales; compared against simple baselines
What would you do with a cluster?	T3 window, XGBoost on Spark, streaming replay, richer repo metadata
Is the model just predicting what is already popular?	Compare to B0; the ablation and lead-time results answer this

13. Deliverables and stretch goals

Final deliverables checklist

- Reproducible pipeline: `make all` runs on the small tier; documented commands for T2.
- Parquet data layers with documented schemas and the row-count funnel.
- Trained model, MLflow runs, and a results table (baselines, ablation, backtest).
- Benchmark results (E1 to E9) with Spark UI screenshots explaining the main differences.
- FastAPI service and dashboard, packaged with Docker.
- Report (about 15 pages) and slide deck; a recorded backup of the live demo.

Stretch goals

- **Streaming replay.** Feed hourly files through Spark Structured Streaming's file source to simulate live scoring, with the leaderboard updating during the demo.
- **Early-adopter score.** Score each user by how often their past stars landed on repos that later broke out (strictly using data before each snapshot), then use the average score of a repo's early stargazers as a feature.
- **Repo metadata.** Add language and topics from the GitHub REST API (rate-limited, so only for top candidates).
- **Text signal.** Embed README text for top candidates to cluster by topic, a natural link to your Gen AI work.
- **Table format upgrade.** Store gold tables as Delta Lake or Iceberg for versioning and time travel.
- **XGBoost on Spark** as an alternative model for comparison against MLlib GBT.

Ethics and compliance

The data is public, but it contains usernames. Publish only repo-level aggregates and model outputs, not profiles of individual users, and review the GH Archive and GitHub terms of use before you share data or results.

Appendix: quick-start order

```
bash scripts/make_urls.sh 2024-01-01 2024-01-07 && aria2c -i urls.txt -d data/raw -j 8 -c   # T0
spark-submit jobs/01_ingest.py "data/raw/2024-01-*.json.gz" data/bronze/events
spark-submit jobs/02_silver.py && spark-submit jobs/03_gold.py
spark-submit jobs/05_features_labels.py && spark-submit jobs/06_train_eval.py
uvicorn api.main:app --reload                                    # then open the dashboard
```